ECE 220 Computer Systems & Programming

Lecture 19 — From C to LC-3 Assembly with Linked Data Structures

October 29, 2026

Prof. Sayan Mitra mitras@illinois.edu
Course website: courses.grainger.illinois.edu/ece220/fa2026

ILLINOIS
Electrical & Computer Engineering
GRAINGER COLLEGE OF ENGINEERING

print_list, Compiled by Hand

```
typedef struct nodeStruct node;
struct nodeStruct {
    char data;
    node *next;
};

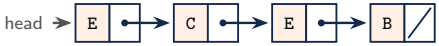
void print_list(node *head) {
    if (head == NULL)
        return;
    printf("%c", head->data);
    print_list(head->next);
}
int main() {
    node *head = &nodes[2];
    print_list(head);
}
```

print_list, Compiled by Hand

```
typedef struct nodeStruct node;
struct nodeStruct {
    char data;
    node *next;
};

void print_list(node *head) {
    if (head == NULL)
        return;
    printf("%c", head->data);
    print_list(head->next);
}
int main() {
    node *head = &nodes[2];
    print_list(head);
}
```

The list is already in memory:



It prints

ECEB

By the end of today you will know what each of the seven lines of `print_list` becomes in LC-3: where `head` lives, what `head->data` is, and how the recursive call finds its way back.

Today's Three Questions

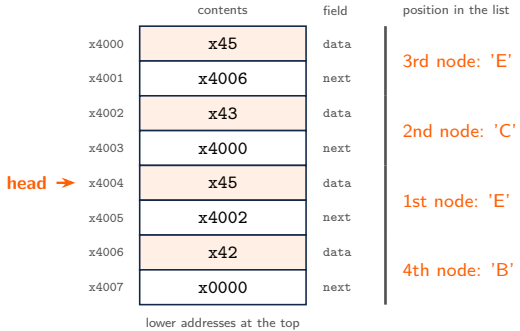
The list sits in memory, and `main` calls `print_list(head)`. What does each line of `print_list` become?

1. `head->data` and `head->next`: which instruction reads a field of a struct?
2. `print_list` has one argument, no locals, and returns nothing. What is in its activation record (AR), and where does R5 point?
3. `printf` becomes `OUT`, which overwrites R7, and the recursive call is a `JSR`. How does each call still return to the right place?

Answers today: one `LDR` per field; the same AR as every function; the return address kept on the run-time stack (RTS).

The List in LC-3 Memory: `data.asm`

```
.ORIG x4000
        .FILL x45               ; x4000  data
        .FILL x4006             ; x4001  next
        .FILL x43               ; x4002  data
        .FILL x4000             ; x4003  next
        .FILL x45               ; x4004  data
        .FILL x4002             ; x4005  next
        .FILL x42               ; x4006  data
        .FILL x0000             ; x4007  next
.END
```



- A node is 2 words: `data` at offset 0, `next` at offset 1. `NULL` is x0000.
- `head` = x4004. Follow `next`: x4004 → x4002 → x4000 → x4006 → NULL
- x45 is 'E', x43 is 'C', x42 is 'B'. Output: ECEB

Each Line, as Instructions

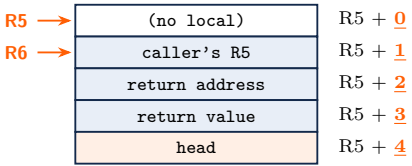
Register plan: R1 holds `head`; R0 holds the character to print.

C	LC-3	Why
`if (head == NULL) return;`	`LDR R1, R5, #4` then `BRz DONE`	the load sets the condition codes; `NULL` is **x0000**
`head->data`	`LDR R0, R1, #0`	a field is an offset from the node's address
`head->next`	`LDR R1, R1, #1`	one `LDR` per field
`printf("%c", head->data);`	`OUT` (`TRAP x21`) with the character in **R0**	the service routine returns through R7, so R7 **changes**
`print_list(head->next);`	push `head->next`, `JSR PRINT_LIST`, pop **2** words	caller set-up, then caller tear-down
`return;` and the closing `}`	`DONE`: callee tear-down, then `RET`	both exits share one tear-down

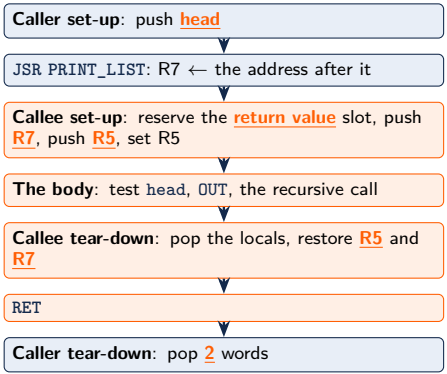
Fields and the NULL test are loads. The call and the return are the calling convention: the AR, next.

The AR of `print_list(head)`

One argument, no locals, nothing returned. The AR still has the three bookkeeping words, and the caller still pops two words after the `JSR`.



- Set-up ends with `ADD R5, R6, #-1`: R5 points at **a word nobody uses**, R6 at **the caller's R5**. Mark both.
- `head` is read with `LDR R1, R5, #4`



main in LC-3, and the Addresses

```
.ORIG x3000
MAIN    LD   R6, USER_STACK    ; SP: R6 <- xFDFF
        ADD  R5, R6, #-1       ; R5: frame pointer
        ADD  R6, R6, #-1       ; main's local: head
        LD   R0, HEAD
        STR  R0, R5, #0        ; head = x4004
; ---- caller set-up: push the argument ----
        LDR  R0, R5, #0        ; R0 <- head
        ADD  R6, R6, #-1
        STR  R0, R6, #0        ; push head
        JSR  PRINT_LIST
; ---- caller tear-down ----
        ADD  R6, R6, #2        ; pop RV and argument
        HALT
HEAD        .FILL x4004
USER_STACK .FILL xFDFF
```

printlist.asm, assembled at x3000; data.asm is loaded first:

```
x3000  MAIN        LD R6, USER_STACK
  ...
x3008              JSR PRINT_LIST
x3009              ADD R6, R6, #2
  ...
x300D  PRINT_LIST  ADD R6, R6, #-1
  ...
x301A              JSR PRINT_LIST
x301B              ADD R6, R6, #2
```

- main's local head is at xFDFE; main pushes x4004 at xFDFD
- main's JSR puts x3009 in R7; the JSR inside PRINT_LIST puts x301B

In-Class Problem: PRINT_LIST, Part 1 — Build the AR

`main` has pushed `head` and jumped to `PRINT_LIST`. Build the AR (`printlist.asm`, lines 21–27).

- **Registers**: R5 frame pointer, R6 stack pointer, R1 `head`, R0 the character
- The return-value slot is reserved although `print_list` returns nothing: the caller's `ADD R6, R6, #2` counts on it
- R7 holds **the return address** right now; it goes into the AR before anything can change it
- No locals: after `ADD R5, R6, #-1` there is nothing more to push

```
; ---- callee set-up ----
PRINT_LIST
        ADD  R6, R6, #-1      ; return-value slot
        ADD  R6, R6, #-1
        STR  R7, R6, #0       ; push return address
        ADD  R6, R6, #-1
        STR  R5, R6, #0       ; push caller's R5
        ADD  R5, R6, #-1      ; R5 -> first local (none)
```

The same six instructions open every function. A function with locals adds one more after them, `ADD R6, R6, #-k`.

In-Class Problem: PRINT_LIST, Part 2 — the Body

- `head` comes from the AR: `LDR R1, R5, #4`; `BRz` tests **head == NULL**
- `head->data` into R0: `LDR R0, R1, #0`, then `OUT`
- `OUT` overwrites R7. Safe, because **the return address is at R5+2**
- The argument of the recursive call is **head->next**: `LDR R1, R1, #1`, push it, `JSR`
- After the `JSR`: pop **2** words. Nothing to read, because `print_list` returns **nothing**

The `JSR` overwrites R7 as well. The tear-down reloads R7 from the AR, so both are undone.

```
; ---- if (head == NULL) return; ----
        LDR  R1, R5, #4        ; R1 <- head
        BRz  DONE              ; head == NULL
; ---- printf("%c", head->data); ----
        LDR  R0, R1, #0        ; R0 <- head->data
        OUT                    ; R7 changes: RA is safe
; ---- print_list(head->next); ----
        LDR  R1, R1, #1        ; R1 <- head->next
        ADD  R6, R6, #-1
        STR  R1, R6, #0        ; push head->next
        JSR  PRINT_LIST
        ADD  R6, R6, #2        ; pop RV and argument
```

In-Class Problem: PRINT_LIST, Part 3 — Tear Down and Run

- `DONE` is reached two ways: from `BRz` when **head == NULL**, and after the recursive call returns
- `ADD R6, R5, #1` pops the locals. Here it changes R6 by **0**
- Then restore **R5**, restore **R7**, leave R6 pointing at **the return value**, `RET`
- Run it with `data.asm` loaded: the monitor shows **ECEB**; `DONE` runs **5** times

The same six instructions tear down every AR. Nothing in them depends on how deep the recursion went.

Draw the RTS at the deepest call, next.

```
; ---- callee tear-down ----
DONE    ADD  R6, R5, #1        ; pop the locals (none)
        LDR  R5, R6, #0        ; restore caller's R5
        ADD  R6, R6, #1
        LDR  R7, R6, #0        ; restore return address
        ADD  R6, R6, #1        ; R6 -> return value
        RET
```

Simulator: `file data`, `file printlist`, `continue`:

```
ECEB
-- halting the LC-3 --
R5=xFDFE R6=xFDFE
```

The RTS at the Deepest Call

		contents	what it is
R5 →	xFDE9		call 5: no local
R6 →	xFDEA	xFDED	caller's R5
	xFDEB	x301B	return address
	xFDEC		return value
	xFDED	x0000	arg head = NULL
	xFDEE	xFDF1	call 4: caller's R5
	xFDEF	x301B	return address
	xFDF0		return value
	xFDF1	x4006	arg head ('B')
	xFDF2	xFDF5	call 3: caller's R5
	xFDF3	x301B	return address
	xFDF4		return value
	xFDF5	x4000	arg head ('E')
	xFDF6	xFDF9	call 2: caller's R5
	xFDF7	x301B	return address
	xFDF8		return value
	xFDF9	x4002	arg head ('C')
	xFDFA	xFDFE	call 1: caller's R5
	xFDFB	x3009	return address
	xFDFC		return value
	xFDFD	x4004	main pushed head
	xFDFE	x4004	main: head

`print_list(NULL)`, the fifth call, has just reached `DONE`. Fill in the RTS; mark R5 and R6.

```
x3000  MAIN        LD R6, USER_STACK
  ...
x3008              JSR PRINT_LIST
x3009              ADD R6, R6, #2
  ...
x300D  PRINT_LIST  ADD R6, R6, #-1
  ...
x301A              JSR PRINT_LIST
x301B              ADD R6, R6, #2
```

ARs on the RTS: **5**, plus `main`'s. x3009 appears **once**; x301B, **4** times. R5 = **xFDE9**, R6 = **xFDEA**.

With no locals the ARs touch: call 4's R5 points at xFDED, call 5's argument. Values from the simulator, stopped at the first arrival at `DONE`.

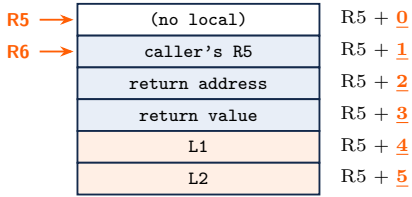
Two Pointer Arguments: `mergeList`

`node *mergeList(node *L1, node *L2)` splices two sorted lists into one. Its recursive case is

```
L1->next = mergeList(L1->next, L2); return L1;
```

with the mirror image when `L2` comes first. What changes when it is compiled?

- Two arguments: the caller pushes `L2` first, then `L1`, so `L1` is at R5+4 and `L2` at R5+5; the AR has 5 words
- The return value is a pointer: the callee stores it at R5+3; the caller reads it from **M[R6]** and writes it into `L1->next` with `STR R0, R1, #1`
- Two call sites, each with its own caller set-up and tear-down; the caller pops 3 words
- Two base cases: `L1 == NULL` returns `L2`, and the other way round; each is a load and a `BRz`



the AR of mergeList(L1, L2)

Lecture 27 converts `mergeList` in full. Everything it needs is on today's slides.

Summary & What's Next

Today

- **The list in memory**: two words per node, `data` at offset 0 and `next` at offset 1; `NULL` is x0000; the nodes need not sit in list order
- **A field is an offset**: `head->data` is `LDR R0, R1, #0`, `head->next` is `LDR R1, R1, #1`; `head == NULL` is a load and a `BRz`
- **The AR with no locals**: caller's R5, return address, return value, `head` at R5+4; R5 points at an unused word, R6 at the caller's R5
- **PRINT_LIST**: the same set-up and tear-down as every function; `OUT` and `JSR` overwrite R7, and the tear-down restores it from R5+2
- **The RTS at the deepest call**: five ARs of four words; every return address is x301B except the first, x3009
- **Two pointer arguments**: R5+4 and R5+5, a pointer in the return-value slot, three words popped by the caller

Next lecture: Introduction to C++, objects and constructors. **Midterm 2 is Thursday, November 5.**

Code from today: `print_list.c`, `data.asm`, `printlist.asm` and `printlist_skel.asm`.